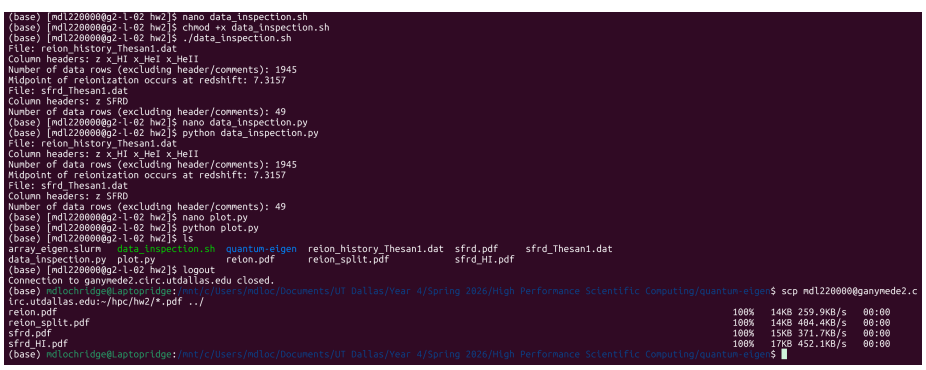
Assignment 2: End-to-End HPC Coding Workflow

Matthew Lochridge

February 2026

1 Moving and Examining Data

A working directory for the assignment was created and Thesan Project data files were downloaded on Ganymede2 using the code provided. A shell script data_inspection.sh and Python script data_inspection.py were written to examine the data by parsing the column headers (ignoring the preceding # and ending parentheticals), counting data rows, and finding the value of x_{HI} corresponding closest to $z \approx 0.5$. Another Python script plot.py was written to visualize the data, and the plots were retrieved from Ganymede2. The following subsections include the inspection and plotting scripts, as they are not included in the quantum-eigen git repository discussed in the next section. Fig. [1] shows the terminal logs for data inspection and plotting. Fig. [2] show the plots generated with plot.py.



```
(base) [mdl220000@g2-l-02 hw2]$ nano data_inspection.sh
(base) [mdl220000@g2-l-02 hw2]$ chmod +x data_inspection.sh
(base) [mdl220000@g2-l-02 hw2]$ ./data_inspection.sh
File: reion_history_Thesan1.dat
Column headers: z x_HI x_HeI x_HeII
Number of data rows (excluding header/comments): 1945
Midpoint of reionization occurs at redshift: 7.3157
File: sfrd_Thesan1.dat
Column headers: z SFRD
Number of data rows (excluding header/comments): 49
(base) [mdl220000@g2-l-02 hw2]$ nano data_inspection.py
(base) [mdl220000@g2-l-02 hw2]$ python data_inspection.py
File: reion_history_Thesan1.dat
Column headers: z x_HI x_HeI x_HeII
Number of data rows (excluding header/comments): 1945
Midpoint of reionization occurs at redshift: 7.3157
File: sfrd_Thesan1.dat
Column headers: z SFRD
Number of data rows (excluding header/comments): 49
(base) [mdl220000@g2-l-02 hw2]$ nano plot.py
(base) [mdl220000@g2-l-02 hw2]$ python plot.py
(base) [mdl220000@g2-l-02 hw2]$ ls
array_eigen.slurm  data_inspection.sh  quantum-eigen  reion_history_Thesan1.dat  sfrd.pdf     sfrd_Thesan1.dat
data_inspection.py  plot.py            reion.pdf      reion_split.pdf            sfrd_HI.pdf
(base) [mdl220000@g2-l-02 hw2]$ logout
Connection to ganymede2.circ.utdallas.edu closed.
(base) mdlochridge@Laptopridge:/mnt/c/Users/mdloc/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/quantum-eigen$ scp mdl220000@ganymede2.c
irc.utdallas.edu:~/hpc/hw2/*.pdf ../
reion.pdf                                                                                                                    100%   14KB 259.9KB/s   00:00
reion_split.pdf                                                                                                              100%   14KB 404.4KB/s   00:00
sfrd.pdf                                                                                                                     100%   15KB 371.7KB/s   00:00
sfrd_HI.pdf                                                                                                                  100%   17KB 452.1KB/s   00:00
(base) mdlochridge@Laptopridge:/mnt/c/Users/mdloc/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/quantum-eigen$
```

Figure 1: Terminal log for inspecting and generating plots from Thesan Project data on Ganymede2 and copying to a local directory.

1.1 data_inspection.sh

#!/bin/bash

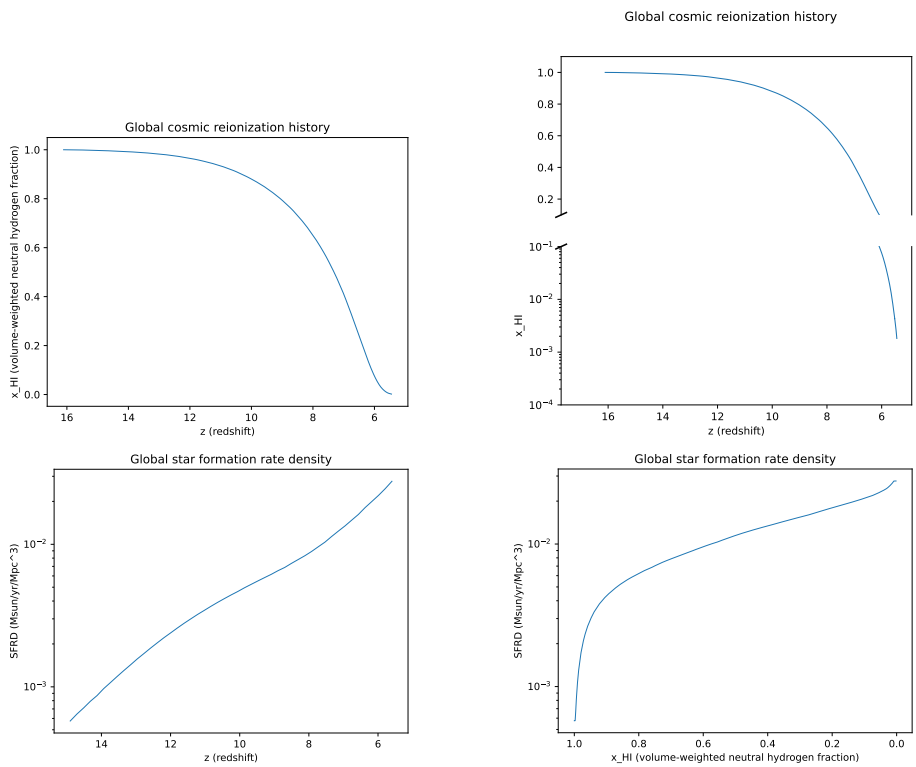


Figure 2: (a) x_{HI} vs. z with linear scales. (b) x_{HI} vs. z with split log-linear y-axis scale. (c) SFRD vs. z. (d) SFRD vs. x_{HI}.

```
set -euo pipefail

get_headers () {
    datafile=$1
    headers=$(head -n 1 "$datafile" | cut -d'#' -f2 | cut -d'[' -f1 | xargs)
    printf "Column headers: %s\n" "$headers"
}

get_num_rows () {
    datafile=$1
    num_rows=$(tail -n +2 "$datafile" | wc -l)
    printf "Number of data rows (excluding header/comments): %d\n" $num_rows
}

get_midpoint_redshift () {
    datafile=$1
    target="0.5"
```

```
    awk -v target="$target" '
    function abs(x){return x<0?-x:x}
    function isnum(s){return s ~ /^[+-]?[0-9]*\.?[0-9]+([eE][+-]?[0-9]+)?$/}
    BEGIN{min=1e99; best_z=""; best_xhi=""}

    !/^[[:space:]]*#/ {
        z=$1; xhi=$2;
        if(!isnum(z) || !isnum(xhi)) next;
        diff = abs(xhi - target);
        if(diff < min){ min = diff; best_z = z; best_xhi = xhi }
    }

    END{
        if(best_z != ""){
            printf("Midpoint of reionization occurs at redshift: %.4f\n", best_z)
        } else {
            print "No valid data rows found in" ARGV[1] > "/dev/stderr";
            exit 1
        }
    }' "$datafile"
}

reion_datafile="reion_history_Thesan1.dat"
printf "File: %s\n" $reion_datafile
get_headers "$reion_datafile"
get_num_rows "$reion_datafile"
get_midpoint_redshift "$reion_datafile"

sfrd_datafile="sfrd_Thesan1.dat"
printf "File: %s\n" $sfrd_datafile
get_headers "$sfrd_datafile"
get_num_rows "$sfrd_datafile"
```

1.2 data_inspection.py

```
def print_headers(file):
    with open(file, 'r') as f:
        headers = f.readline().split('[')[0].strip().split('#')[1]
    print(f'Column headers: {headers}')

def count_rows(file):
    with open(file, 'r') as f:
        next(f)
        row_count = sum(1 for line in f)
```

```
        print(f'Number of data rows (excluding header/comments): {row_count}')

def midpoint_redshift(file):
    with open(file, 'r') as f:
        next(f)
        for line in f:
            data = line.strip().split(' ')
            z = float(data[0])
            x_HI = float(data[1])
            if x_HI <= 0.5:
                if prev_x_HI-0.5 < 0.5-x_HI:
                    print(f'Midpoint of reionization occurs at redshift: {prev_z}')
                else:
                    print(f'Midpoint of reionization occurs at redshift: {z}')
                    break
            prev_z = z
            prev_x_HI = x_HI

reion_file = 'reion_history_Thesan1.dat'
print('File: ' + reion_file)
print_headers(reion_file)
count_rows(reion_file)
midpoint_redshift(reion_file)

sfrd_file = 'sfrd_Thesan1.dat'
print('File: ' + sfrd_file)
print_headers(sfrd_file)
count_rows(sfrd_file)
```

1.3 plot.py

```
import numpy as np
import matplotlib.pyplot as plt

def load_table(fname):
    data = np.loadtxt(fname, comments="#")
    z = data[:, 0]
    vals = data[:, 1]
    # ensure increasing z for interpolation
    order = np.argsort(z)
    return z[order], vals[order]

def main():
    z_hi, xHI = load_table("reion_history_Thesan1.dat")
```

```
z_sfrd, sfrd = load_table("sfrd_Thesan1.dat")

# reion history: xHI vs z
plt.figure()
plt.plot(z_hi, xHI, lw=1)
plt.xlabel("z (redshift)")
plt.ylabel("x_HI (volume-weighted neutral hydrogen fraction)")
plt.title("Global cosmic reionization history")
plt.gca().invert_xaxis()
plt.savefig("reion.pdf", bbox_inches="tight")
plt.close()

# split y-axis: linear [0.1,1] on top, log [1e-4,0.1] on bottom
fig, (ax_top, ax_bot) = plt.subplots(
    2,
    1,
    gridspec_kw={"height_ratios": [1, 1]},
    figsize=(6, 6),
)
ax_top.plot(z_hi, xHI, lw=1)
ax_top.set_xlim(0.9*min(z_hi), 1.1*max(z_hi))
ax_top.set_ylim(0.1, 1.1)
ax_top.set_yscale("linear")
ax_top.spines["bottom"].set_visible(False)
ax_top.xaxis.set_ticks([])

ax_bot.plot(z_hi, xHI, lw=1)
ax_bot.set_xlim(0.9*min(z_hi), 1.1*max(z_hi))
ax_bot.set_ylim(1e-4, 0.1)
ax_bot.set_yscale("log")
ax_bot.spines["top"].set_visible(False)

# diagonal break markers
d = 0.015
kwargs = dict(transform=ax_top.transAxes, color="k", clip_on=False)
ax_top.plot((-d, +d), (-d, +d), **kwargs)
kwargs = dict(transform=ax_bot.transAxes, color="k", clip_on=False)
ax_bot.plot((-d, +d), (1 - d, 1 + d), **kwargs)

ax_bot.set_xlabel("z (redshift)")
ax_bot.set_ylabel("x_HI")
ax_top.invert_xaxis()
ax_bot.invert_xaxis()
fig.suptitle("Global cosmic reionization history")
plt.savefig("reion_split.pdf", bbox_inches="tight")
plt.close()
```

```
    # SFRD vs z (use log scale for y)
    plt.figure()
    plt.plot(z_sfrd, sfrd, lw=1)
    plt.xlabel("z (redshift)")
    plt.ylabel("SFRD (Msun/yr/Mpc^3)")
    plt.yscale("log")
    plt.title("Global star formation rate density")
    plt.gca().invert_xaxis()
    plt.savefig("sfrd.pdf", bbox_inches="tight")
    plt.close()

    # Map SFRD onto xHI by interpolating SFRD to the redshifts of xHI
    sfrd_on_hi = np.interp(z_hi, z_sfrd, sfrd)
    plt.figure()
    plt.plot(xHI, sfrd_on_hi, lw=1)
    plt.xlabel("x_HI (volume-weighted neutral hydrogen fraction)")
    plt.ylabel("SFRD (Msun/yr/Mpc^3)")
    plt.yscale("log")
    plt.title("Global star formation rate density")
    plt.gca().invert_xaxis()
    plt.savefig("sfrd_HI.pdf", bbox_inches="tight")
    plt.close()

if __name__ == "__main__":
    main()
```

2 GitHub Repository Setup

A public GitHub repository was set up for the remainder of the assignment. The link to the repository is https://github.com/Matthew-Lochridge/quantum-eigen. The remaining sections omit code since it is all available to view on GitHub.

The uppermost repository structure consists of a README, LICENSE, src folder, tests folder, slurm batch script, and Jupyter notebook for analysis. Locally, there is also a results folder to store solutions and a plots folder to store figures. The src folder contains the 2D Hamiltonian solver. The tests folder contains unit tests. The results folder contains output files, including files for the lowest N eigenvalues and files for the ground state probability density, named for each N and potential. The plots folder contains figures generated from the analysis notebook.

The choices to contain some files within subfolders was made to more easily distinguish output files from code, to keep all the tests in one suite and run them all serially with one command, and to contain code called by other files within its own src folder. Additionally, output files from computation are stored

in results separate from image files stored in plots. Since the slurm batch script and analysis notebook do not fall into any of these categories, they were left at the highest level directory. The file structure is intended for all files to be run from the highest level directory quantum-eigen.

Local unit tests within the tests directory can be called with the command python -m pytest tests

The solver script can be called locally with the command
python src/eigen.py −−N −−potential −−n_eigs −−bc −−save_gs

The slurm batch script can be called on Ganymede2 with the command
sbatch array_eigen.slurm

3 Local Development of a 2D Hamiltonian Solver

The 2D Hamiltonian solver script eigen.py was built from the provided code. The signs of the derivative terms in the Hamiltonian were changed to better reflect analytical solutions. Fig. [3] shows the terminal log for unit tests and a local run.

A sinusoidal potential was added as an option, with the form

$$V_{\text{sinusoidal}} = A \sin(\pi x) \sin(\pi y) \tag{1}$$

where $A = 100$ to illustrate the anisotropy clearly.

A Boolean option to save the ground state probability density was added. These data are saved in a separate file in results with the naming format gs_prob_N_potential.txt.

Dirichlet boundary conditions of the form

$$\psi(x, y) = ax + by \tag{2}$$

were added as an option, with $a = b = 0.003$. This slightly decreases runtime by reducing the unknown grid points from N^2 to $(N-2)^2$. Furthermore, the ground state probability density is iteratively smoothened by averaging neighbors to minimize sharpness at the boundaries. Boundary conditions were originally implemented in the git branch boundary-conditions, which was later merged into main.

The argparse package was imported to utilize command line arguments to set the grid dimension N, select the potential, select the number of eigenvalues n_{eigs} to report, choose to save the ground state probability density, and choose to include boundary conditions.

Sanity checks were imposed on the arguments: to enforce proper data types (i.e. positive integer for N and n_{eigs}, string for potential and boundary conditions, and bool for optionally saving ground state probability density); to ensure that $n_{\text{eigs}} \leq N^2$; to only select potentials and boundary conditions that are implemented (i.e. well, harmonic, or sinusoidal potentials and Dirichlet boundary conditions); and to ensure that $N \geq 3$ when using boundary conditions so that there are interior points to be determined.

The example unit test was included in test_simple.py. Four additional unit tests were added to this file: to check that the matrix size is reduced when using boundary conditions: to check that the ground state probability density is normalized and nonnegative everywhere (two separate tests); and to check that the smoothing iteration converges.

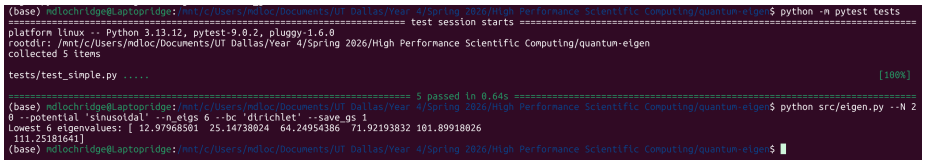


Figure 3: Terminal log showing unit tests passed and a local run using command line arguments.

4 Running a Larger Problem on an HPC Cluster

The slurm batch script was modified from the provided example to include the sinusoidal potential and command line arguments, to extend the array of N values, and to extend the time to 15 minutes. N was set to run up to 10240, however $N > 80$ did not process within the time limit. Fig [4] shows the terminal log for submitting a batch job and retrieving results.

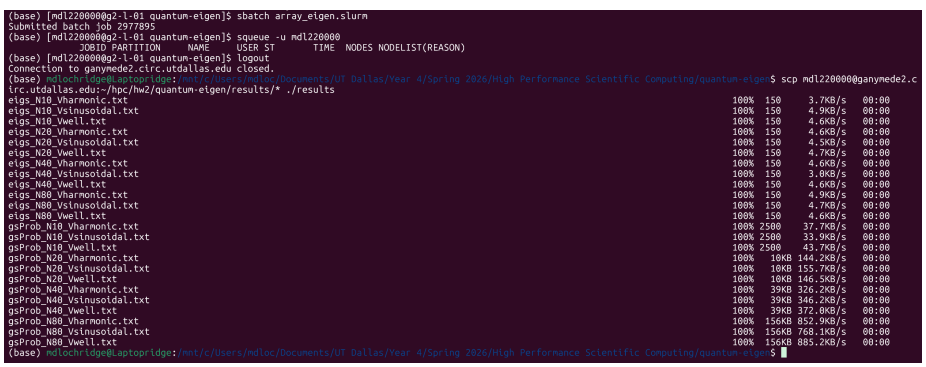


Figure 4: Terminal log showing batch run on Ganymede2 and copy of results to local directory.

5 Local Visualization and Analysis

Local visualization of the Ganymede2 computation was done in the Jupyter notebook analysis.ipynb. The example code was modified to loop through each

potential for Figs. [5] and [6].

Fig. [5] shows the lowest six eigenvalues for each potential with $N = 80$, the finest grid attempted that would process on Ganymede2 within the time limit. Analytical solutions are also shown for the well and harmonic potentials for comparison. Evidently the degeneracy seen with the well and harmonic potentials is broken by the anisotropy of the sinusoidal potential.

Fig. [6] shows the potential landscapes. Additional code was written to plot the ground state probability density computed on Ganymede for $N = 80$, shown in Fig. [7]. As expected, the ground state probability density for the isotropic well and harmonic potentials is also isotropic and concentrated near the center. Furthermore, the sinusoidal potential and corresponding ground state probability density show the same mirror symmetry along the diagonals, with higher probability density concentrated around the low-potential quadrants and vanishing probability density around the high-potential quadrants.

Fig. [8] shows how each of the lowest eigenvalues changes with N for each potential. Evidently numerical solutions converge toward analytical solutions much faster for the well potential than for the harmonic potential.

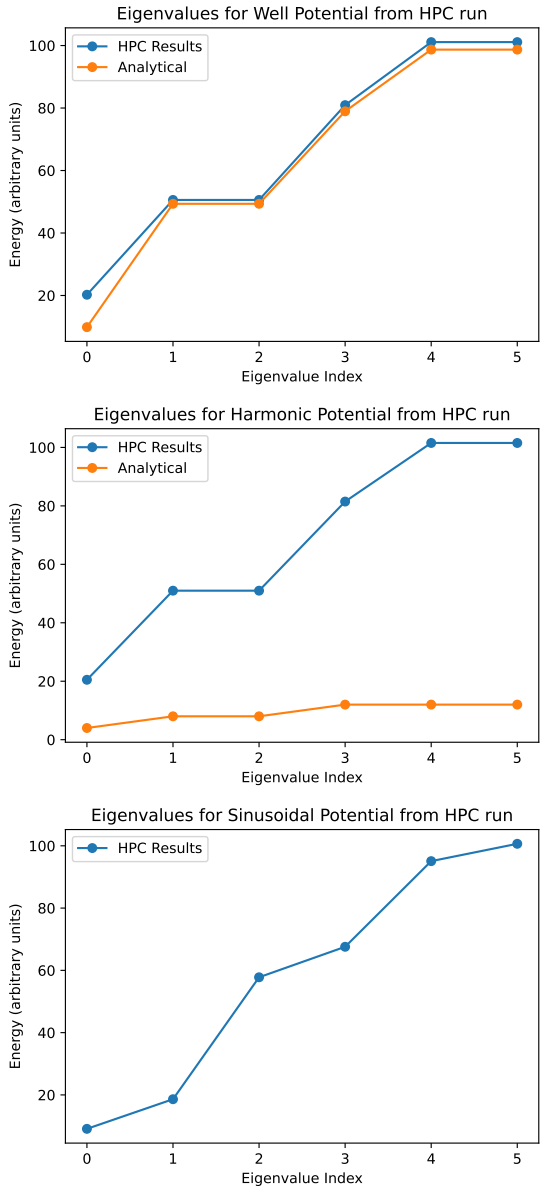


Figure 5: Eigenvalues from Ganymede2 runs with $N = 80$. Analytical solutions are also shown for the well and harmonic potentials.

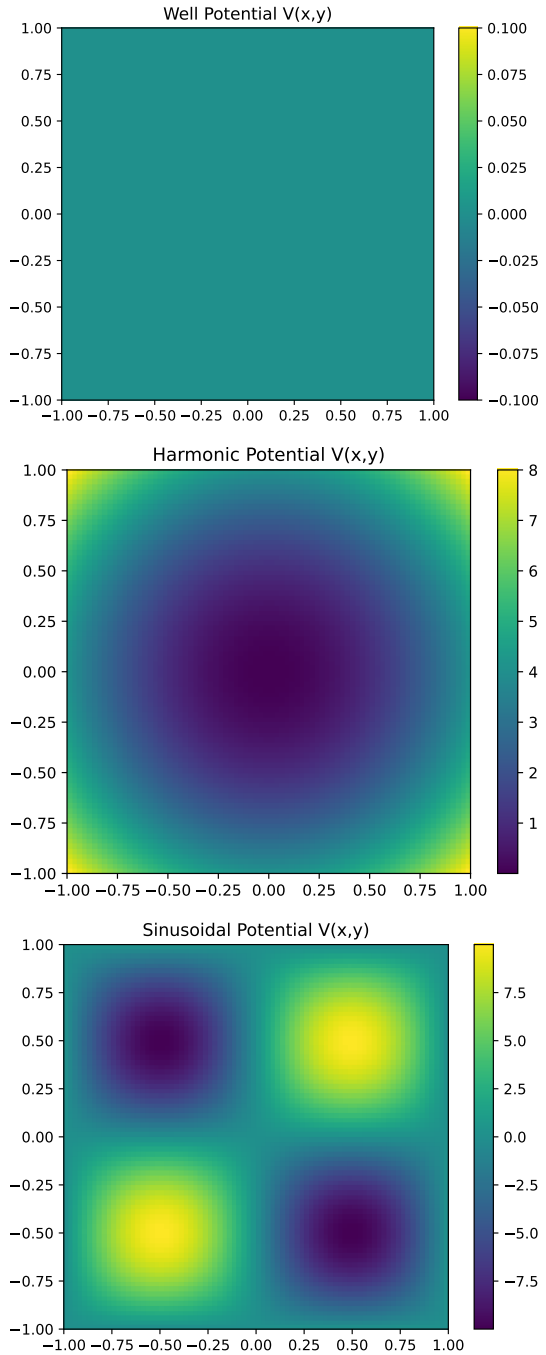


Figure 6: Potential landscapes with $N = 80$.

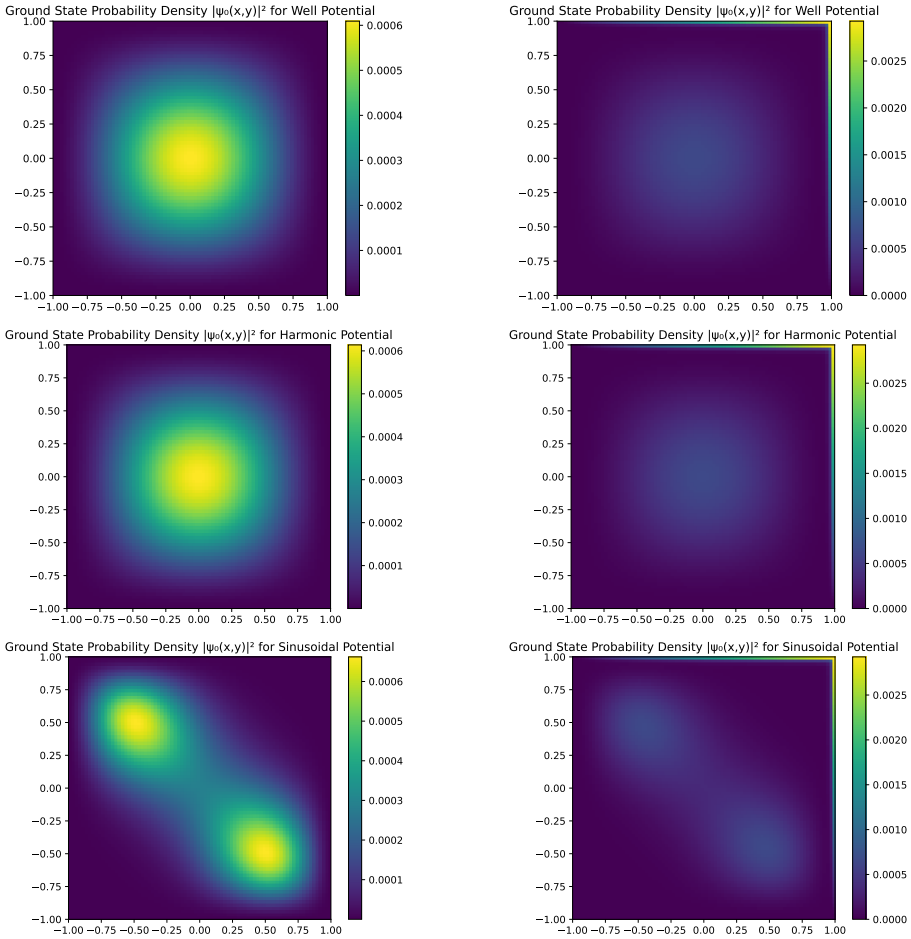


Figure 7: Ground state probability densities from Ganymede2 run with $N = 80$. The left images were produced without boundary conditions, and the right images were produced with the boundary conditions described in the previous section.

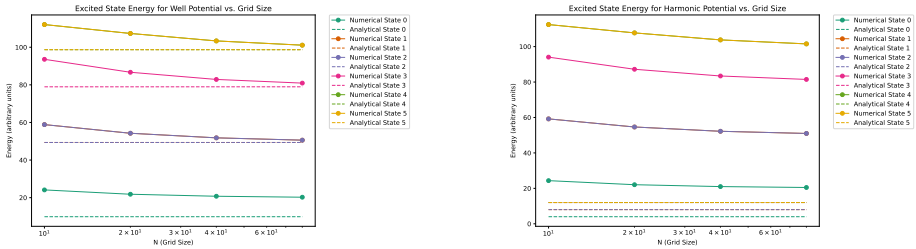


Figure 8: Eigenvalues vs. N from Ganymede2 run with the well potential and harmonic potential. Analytical solutions are shown as dashed horizontal lines.